

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Операционные системы и системное программирование

ОТЧЕТ

по лабораторной работе № 4

Тема: «Задача производителя-потребителя для процессов»

Выполнил:
студент гр. 450502 Бурш А.П.

Проверил:
Благиных В.Д.

Минск, 2026

1 ЦЕЛЬ ЛАБОРАТОРНОЙ РАБОТЫ

Изучить задачу взаимодействия процессов «производитель-потребитель» с использованием разделяемой памяти и семафоров System V.

Реализовать основной процесс, который создает очередь сообщений и по командам пользователя порождает/завершает процессы двух типов: producers и consumers.

Очередь сообщений должна быть реализована как кольцевой буфер с индексами head/tail и счетчиками: добавленных, извлеченных и текущего заполнения.

Процессы-производители формируют сообщения и помещают их в очередь только при наличии свободного места.

Процессы-потребители извлекают сообщения только при их наличии, обрабатывают данные и проверяют контрольную сумму.

Синхронизация доступа к очереди выполняется через три семафора: MUTEX, FREE_SLOTS и QUEUED_ITEMS.

Формат сообщения: type (1 байт), hash (2 байта), size (1 байт), data длиной $((size + 3) / 4) * 4$ байт.

Поля size и data генерируются случайно; при вычислении hash поле hash предварительно принимается равным нулю, а расчет ведется по size + 1 байтам сообщения.

После помещения сообщения producer увеличивает счетчик addedCount, после извлечения consumer увеличивает extractedCount и выводит актуальное значение счетчика.

Следует обеспечить корректное завершение по сигналам, наличие задержек для наблюдения за работой и защиту от тупиковых ситуаций при отсутствии producer или consumer.

Интерфейс управления должен позволять смотреть текущее состояние системы: число producers/consumers и заполненность очереди.

2 ВЫПОЛНЕНИЕ РАБОТЫ

2.1 Описание алгоритма выполнения работы

Основной процесс создает IPC-ресурсы (shared memory + набор семафоров), после чего в цикле обрабатывает команды пользователя. По командам запускаются и завершаются дочерние процессы двух типов. Производители вызывают `produceMessage()`, ждут свободный слот, получают `mutex` и добавляют сообщение в кольцевую очередь. Потребители ждут появления сообщений, получают `mutex`, извлекают сообщение и проверяют `hash` через `consumeMessage()`. Функции `semDown()/semUp()` обеспечивают корректную синхронизацию и предотвращают доступ к пустой или переполненной очереди. При выходе `deleteSemaphores()` завершает дочерние процессы и очищает IPC-объекты.

2.2 Описание основных функций

Программа состоит из нескольких частей, представляющих собой некоторые функции. К ним относятся функции:

- 1) Запуск программы;
- 2) Создание потребителя;
- 3) Удаление потребителя;
- 4) Потребление сообщения;
- 5) Создание производителя;
- 6) Удаление производителя;
- 7) Производство сообщения;
- 8) Захват семафора;
- 9) Освобождение семафора;
- 10) Вычисление контрольных данных сообщения;
- 11) Инициализация очереди сообщений;
- 12) Вывод статуса очереди сообщений;
- 13) Инициализация семафоров;

2.2.1 Функция `runApp()`

Основной управляющий цикл программы: инициализирует IPC-ресурсы, обрабатывает команды меню и выполняет корректное завершение через `deleteSemaphores()`.

2.2.2 Функция `createProducer()`

Создает дочерний процесс-производитель, добавляет его PID в стек `producers` и запускает цикл генерации сообщений.

2.2.3 Функция `deleteProducer()`

Удаляет последнего производителя из стека: отправляет SIGTERM и ожидает завершение через waitpid().

2.2.4 Функция produceMessage()

Формирует сообщение: генерирует поля type/size/data и вычисляет контрольную сумму hash.

2.2.5 Функция createConsumer()

Создает дочерний процесс-потребитель, добавляет его PID в стек consumers и запускает цикл извлечения сообщений.

2.2.6 Функция deleteConsumer()

Удаляет последнего потребителя из стека: отправляет SIGTERM и ожидает завершение через waitpid().

2.2.7 Функция consumeMessage()

Проверяет целостность сообщения: пересчитывает hash и сравнивает с исходным значением.

2.2.8 Функция calculateHash()

Вычисляет 16-битную контрольную сумму сообщения по первым size + 1 байтам структуры Message.

2.2.9 Функция initQueue()

Инициализирует кольцевую очередь сообщений: обнуляет счетчики, индексы head/tail и буфер.

2.2.10 Функция semDown()

Операция P над семафором System V (уменьшение на 1) с блокировкой процесса при недоступности ресурса.

2.2.11 Функция semUp()

Операция V над семафором System V (увеличение на 1), сигнализирующая о доступности ресурса.

2.2.12 Функция initSemaphores()

Создает/подключает shared memory для очереди и инициализирует набор семафоров MUTEX, FREE_SLOTS, QUEUED_ITEMS.

2.2.13 Функция deleteSemaphores()

Завершает дочерние процессы и удаляет IPC-ресурсы: shared memory и семафоры.

3 ОПИСАНИЕ ПОРЯДКА СБОРКИ

Проект собирается с помощью CMake. В корневом каталоге проекта команда `cmake -S . -B build && cmake --build build` создает исполняемый файл `build/lab4` из исходников в `src/` и заголовочных файлов в `include/`.

Запуск выполняется командой `./build/lab4`. Для очистки результатов сборки используется `cmake --build build --target clean` (или полная очистка: `rm -rf build`).

4 РЕЗУЛЬТАТ РАБОТЫ ПРОГРАММЫ

```
rkuzm@archlinux] [/dev/pts/1]
L[~/Programming/OSaSP/lab4/build]> ./semaphores

=== Producer/Consumer Menu ===
Status: producers=0, consumers=0, queue=0
1. Add producer
2. Remove producer
3. Add consumer
4. Remove consumer
5. Show runtime stats
0. Exit
Select action [0-5]: 1
Producer (PID=44665) created successfully. Current producers count: 1

=== Producer/Consumer Menu ===
Status: producers=1, consumers=0, queue=0
1. Add producer
2. Remove producer
3. Add consumer
4. Remove consumer
5. Show runtime stats
0. Exit
Select action [0-5]: Pid: 44665 produce message: type = 1A | hash = 99F9 | size = 024 | addedCount = 1
1
Producer (PID=44677) created successfully. Current producers count: 2

=== Producer/Consumer Menu ===
Status: producers=2, consumers=0, queue=1
1. Add producer
2. Remove producer
3. Add consumer
4. Remove consumer
5. Show runtime stats
0. Exit
Select action [0-5]: Pid: 44677 produce message: type = 2C | hash = 01AB | size = 243 | addedCount = 2
Pid: 44665 produce message: type = 3A | hash = 9E7F | size = 127 | addedCount = 3
Pid: 44677 produce message: type = 31 | hash = F433 | size = 234 | addedCount = 4
Pid: 44665 produce message: type = 0F | hash = 663C | size = 104 | addedCount = 5
Pid: 44677 produce message: type = 0D | hash = 497B | size = 241 | addedCount = 6
2
Producer (PID=44677) removed successfully. Current producers count: 1

=== Producer/Consumer Menu ===
Status: producers=1, consumers=0, queue=6
1. Add producer
2. Remove producer
3. Add consumer
4. Remove consumer
5. Show runtime stats
0. Exit
Select action [0-5]: 2
Producer (PID=44665) removed successfully. Current producers count: 0
```

```

=== Producer/Consumer Menu ===
Status: producers=0, consumers=0, queue=6
1. Add producer
2. Remove producer
3. Add consumer
4. Remove consumer
5. Show runtime stats
0. Exit
Select action [0-5]: 5

Runtime queue stats:
Added messages: 6
Extracted messages: 0
Current queue size: 6
Buffer head index: 6
Buffer tail index: 0

```

Рисунок 1 – Создание и удаление нескольких процессов-производителей и вывод статуса очереди сообщений

```

=== Producer/Consumer Menu ===
Status: producers=0, consumers=0, queue=6
1. Add producer
2. Remove producer
3. Add consumer
4. Remove consumer
5. Show runtime stats
0. Exit
Select action [0-5]: 3
Consumer (PID=48748) created successfully. Current consumers count: 1

=== Producer/Consumer Menu ===
Status: producers=0, consumers=1, queue=6
1. Add producer
2. Remove producer
3. Add consumer
4. Remove consumer
5. Show runtime stats
0. Exit
Select action [0-5]: Pid: 48748 consume message: type = 45 | hash = 37C9 | size = 118 | extractedCount = 1
3
Consumer (PID=48749) created successfully. Current consumers count: 2

=== Producer/Consumer Menu ===
Status: producers=0, consumers=2, queue=5
1. Add producer
2. Remove producer
3. Add consumer
4. Remove consumer
5. Show runtime stats
0. Exit
Select action [0-5]: Pid: 48749 consume message: type = 12 | hash = 7B1E | size = 213 | extractedCount = 2
Pid: 48748 consume message: type = 26 | hash = 9909 | size = 050 | extractedCount = 3
Pid: 48749 consume message: type = 0D | hash = CB85 | size = 210 | extractedCount = 4
Pid: 48748 consume message: type = 00 | hash = AB15 | size = 199 | extractedCount = 5
Pid: 48749 consume message: type = 03 | hash = DC0F | size = 192 | extractedCount = 6
4
Consumer (PID=48749) removed successfully. Current consumers count: 1

=== Producer/Consumer Menu ===
Status: producers=0, consumers=1, queue=0
1. Add producer
2. Remove producer
3. Add consumer
4. Remove consumer
5. Show runtime stats
0. Exit
Select action [0-5]: 4
Consumer (PID=48748) removed successfully. Current consumers count: 0

```

```
=== Producer/Consumer Menu ===  
Status: producers=0, consumers=0, queue=0  
1. Add producer  
2. Remove producer  
3. Add consumer  
4. Remove consumer  
5. Show runtime stats  
0. Exit  
Select action [0-5]: 5  
  
Runtime stats:  
Producers: 0  
Consumers: 0  
Queue: 0/100  
Created messages: 6  
Received messages: 6
```

Рисунок 2 – Создание нескольких процессов-потребителей, вывод статуса очереди сообщений и выход из программы

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы реализована модель «производители-потребители» для процессов с использованием кольцевой очереди, shared memory и семафоров System V.

Получена корректная синхронизация доступа к общей очереди: процессы запускаются и завершаются из CLI, а обмен сообщениями выполняется без нарушения целостности данных.